Computación gráfica y ambientes virtuales (0494)

Práctico 06

Objetivo

El objetivo es que el usuario pueda **interactuar con la escena** y que la aeronave se mueva según un modelo físico. Para eso hay que sincronizar en el tiempo dos funcionalidades distintas del proyecto: la generación y pintado de los cuadros (frames) y el comportamiento de la aeronave (FDM). En el primer caso —cuánto demora generar y dibujar un cuadro— depende de la **máquina** y de la carga que tenga el sistema operativo y es un tiempo real (físico), mientras que en el segundo —cómo se avanza el tiempo de la **simulación**— existe un control parcial porque se pueden ejecutar pasos discretos de avance en el tiempo ya que el FDM no se ejecuta a *tiempo real*, pero requiere que esté sincronizado con el pintado de los cuadros. A esto hay que sumarle, además, que el usuario pueda interactuar con la escena por medio de uno o varios dispositivos de entrada.

Punto de partida

Se puede partir tanto de la escena del Práctico 05, con la cámara orbital, como de la del Práctico 04. No es necesario realizar la implementación del modelado de la dinámica de cuerpo rígido de una aeronave (no es contenido de esta materia) por lo que se provee el módulo con el modelo de dinámica de vuelo (*FDM - Flight Dynamics Model*) para incorporarlo al proyecto. Se puede obtener a partir del siguiente repositorio (como respaldo, en el aula virtual hay un `.zip` con el mismo contenido, que se descomprime dentro de `libs/` para que quede en `libs/dlfdm`):

```
git clone https://github.com/dllorens/CGyAV-dlfdm.git libs/dlfdm
cd libs/dlfdm
make                 # compila el modulo y un ejecutable de prueba
./build/dlfdm-test   # 60 segundos de simulacion que se ejecutan off-line
```

El módulo se provee con un `main` que genera un ejecutable de prueba en donde se puede simular la dinámica de movimiento de la aeronave para distintas condiciones iniciales y para diferentes entradas de comandos. Por defecto corre una condición inicial de equilibrio en vuelo recto y nivelado a 5000 [m] y a una velocidad de $\approx$ 540 [km/h] (291 kn), con una oscilación residual en altura de $\pm$20 [m]. Conviene correr este ejecutable de prueba **antes** de integrar el módulo al proyecto para verificar que compila y resuelve el movimiento de la aeronave de forma correcta.

La integración al proyecto se hace con el mismo mecanismo que ya está en uso para la librería `glad`: el módulo declara sus fuentes e *includes* en un archivo `.mk` que, luego, se incluye vía el `Makefile` del proyecto. Ejemplo a continuación:

```
DLFDM_DIR = ./libs/dlfdm
include $(DLFDM_DIR)/dlfdm.mk    # (1) las dos lineas, junto al bloque de GLAD_DIR

INC_DIRS = \
    $(GLAD_INC_DIRS) \
    $(DLFDM_INC_DIRS) \
    ./src                        # (2) la linea de arriba es la que se agrega

LIB_DIRS = \
    $(GLAD_LIB_DIRS) \
    $(DLFDM_LIB_DIRS)            # (3)
```

Parte 1 — El *game loop* con acumulador

Si en el Práctico 03 se planteara una animación de las figuras geométricas, avanzando con un incremento **por cuadro** alguna variable de posición, como por ejemplo un ángulo (`angulo += 0.01f`), la ejecución de la escena puede verse diferente en distintas máquinas. Por ejemplo, en un equipo que dibuja 60 cuadros por segundo, la figura gira a 0,60 rad/s mientras que en uno que dibuja 144 cuadros por segundo, el giro tiene una velocidad de 1,44 rad/s, a pesar de que el código de la animación es el mismo.

Hay dos situaciones que dependen del tiempo en la escena que requieren estrategias diferentes:

- **Animaciones** —una hélice que gira, la cámara que se mueve—: se soluciona multiplicando por la duración del cuadro: `angulo += velocidad * frame_dt`. Con eso el movimiento deja de depender de la capacidad de la máquina.
- **Simulación** de la dinámica de la aeronave: requiere un enfoque distinto porque el modelo se integra y el error de cada paso crece con el tamaño del paso. Necesita un paso **pequeño y constante** para que los resultados sean consistentes (no puede depender del tiempo que demoran en generarse los cuadros).

Una forma de sincronizar estas dos situaciones de la aplicación es usar la estrategia de **paso fijo con acumulador**: se acumula el tiempo que demora cada cuadro en generarse (y pintarse) y después se consume ese acumulador en pasos discretos dependientes del tamaño de paso de tiempo que usa el FDM.

```
// punto de partida, no molde
const double dt = 1.0 / 120.0;            // el paso del FDM, fijo
double antes    = glfwGetTime();
double acc      = 0.0;

while (!glfwWindowShouldClose(window)) {
    glfwPollEvents();

    double ahora    = glfwGetTime();
    double frame_dt = ahora - antes;
    antes = ahora;

    // ... leer la entrada del usuario (Parte 2)

    acc += std::min(frame_dt, 0.25);      // el tope corta la divergencia del acumulador
                                          // (std::min pide <algorithm>)
    while (acc >= dt) {
        // ... un paso de simulacion (Parte 3)
        acc -= dt;
    }

    // ... actualizar escena y camara, y dibujar
}
```

El resto que queda en `acc` al terminar el cuadro **no se descarta**: es tiempo que todavía no se simuló y queda para el cuadro siguiente. Mientras no se pierda este tiempo remanente se cumple que *el tiempo simulado más el resto del acumulador es igual al tiempo de reloj.*

Algunas cuestiones a pensar.

- En el **primer cuadro** el acumulador vale cero. ¿Cuántos pasos de simulación corre ese cuadro? ¿Está bien que sea así?
- ¿Qué pasa si un cuadro tarda mucho —al arrastrar la ventana, o al volver de una pausa en el depurador— y **no** está el límite para el valor del acumulador?
- La lectura de la entrada del usuario queda **afuera** del `while` interno. ¿Por qué no adentro?

Parte 2 — Ampliar `InputHandler` con entradas de teclado

El módulo desarrollado en el Práctico 05 hay que ampliarlo para que procese entradas de teclado. Ahora tiene que producir dos cosas: un comando de cámara, que ya hacía, y las posiciones de los comandos de la aeronave que consume el modelo de dinámica de vuelo. Además, va a hacer falta un método para fijar el **estado inicial** de esas posiciones de comando, que pueden iniciarse en un valor distinto de cero (se usa en la Parte 3 del práctico).

```
// InputHandler.h --- punto de partida, no molde
class InputHandler {
public:
    void update(GLFWwindow* window, float dt);        // polling, 1 vez por cuadro
    const dlfdm::ControlInputs& controls() const;     // lo que consume el FDM
    void set_controls(const dlfdm::ControlInputs& c); // condicion inicial (Parte 3)
    CameraCommand camera_cmd() const;                 // del Practico 05
};
```

¿Estado o evento?

La elección entre usar la estrategia de *polling* o *callback* para capturar las entradas del usuario hay que decidirla por la naturaleza de la acción que se quiere representar. Cuando el control que se busca es *continuo* —los cuatro mandos, la cámara— hay que aplicar *polling* una vez por cuadro (se pregunta *¿está apretada ahora?*, que es un **estado**). Cuando se quiere representar una acción discreta —pausar, alternar el modo wireframe, cambiar de vista— se resuelve por *callback* (se pregunta *¿se acaba de apretar?*, que es un **evento**).

Rampa para los comandos

Al tratar de emular el comportamiento de los comandos de la aeronave —que son proporcionales— usando como método de entrada el teclado —que es discreto—, conviene que la tecla fije la **velocidad de cambio** del comando y no el valor de la deflexión. Esto aproxima mejor cómo siente un piloto el comportamiento de la aeronave. El lado negativo es que se pierde el comportamiento natural del sistema de comando de una aeronave, en el cual al soltar la palanca, esta vuelve sola a una posición de equilibrio en donde el piloto no tiene que hacer fuerza. En cambio con la emulación de teclado, la deflexión queda en un valor fijo y hay que "centrar" manualmente el comando (no realista pero difícil de resolver).

```
// mientras la tecla este apretada, el comando (deflexion) CAMBIA a una velocidad dada
if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
    eje_ += rate * dt;          // velocidad en [1/s]; eje_ normalizado a [-1, +1]

// ... primero acotar eje_ , luego convertirlo a radianes con el tope de la superficie
```

Relación entre las entradas de teclado y los comandos de la aeronave

Los comandos de la aeronave son tres ejes con signo —elevador, alerones y timón de dirección; el comando de potencia se encuentra normalizado entre 0 y 1 y no requiere analizar el signo—, o sea, hay ocho combinaciones posibles y el problema es que todas estas combinaciones hacen que el avión se mueva pero de una manera que no se ajusta con la realidad física. Para determinar cuáles son los signos correctos para las deflexiones de los comandos hay que analizar la física del problema (fuera del alcance de esta materia), por lo que hay que "aceptar" sin demostración que son los que se muestran en la tabla a continuación:

mando	movimiento del comando	respuesta de la aeronave	deflexión
Elevador	se empuja el bastón	baja la nariz	$\delta_e > 0$
Alerones	bastón a la derecha	ala derecha abajo	$\delta_a > 0$
Timón	pedal derecho	la nariz va a la derecha	$\delta_r < 0$

Lo que la tabla no informa, y hay que decidir, es qué espera el usuario de la tecla: si la flecha es el bastón —adelante, nariz abajo, como en un avión— o si la flecha es la nariz del avión. Ninguna de

las dos es más correcta que la otra, pero hay que elegir una y dejar registrada la decisión: la línea de código es idéntica en los dos casos y sólo cambia qué creía estar implementando quien la escribió. Como sugerencia se recomienda seguir la convención de que la tecla representa el bastón para que alguien acostumbrado a volar obtenga el mismo comportamiento que en una aeronave.

Parte 3 — El FDM en el *loop*

El FDM se encarga de realizar la integración de las variables de estado del problema; esto se realiza en cada llamada a `update()`: se avanza el estado un paso de tiempo Δt y se guarda el resultado de la integración. Es por esto que el método `update()` va adentro del bucle `while` de paso fijo; sin embargo, para recuperar la posición y la actitud **ya integradas**, hay que hacer una llamada al método `getState()` por fuera del bucle una sola vez por cuadro ya que para dibujar se utiliza el último estado calculado para la aeronave.

El modelo de la aeronave

El FDM sólo resuelve el conjunto de ecuaciones que describen el comportamiento de una aeronave por lo que hace falta pasarle la descripción física de la misma. Esto permite que el mismo solver se pueda utilizar para diferentes aeronaves: un planeador, un avión a hélice, un entrenador jet, etc. Esta descripción se construye mediante un conjunto de parámetros —masa, inercias, datos de referencia del ala, coeficientes aerodinámicos, características del motor, topes de deflexión de las superficies, entre otros— que se encuentran agrupados en una estructura denominada `AircraftParameters`.

El repositorio trae un modelo de aeronave listo para usar que se encuentra en el archivo de encabezado `models/aircraft/jettrainer.h`. En este archivo hay una función denominada `jettrainer::load_model()` que devuelve el set de parámetros para inicializar el FDM.

La condición inicial

La simulación debe iniciar desde una condición inicial que será, para el práctico, una condición de equilibrio —*trim*— para que la aeronave esté estabilizada y el usuario no tenga que intervenir inmediatamente para controlarla. La condición de *equilibrio* se compone de un par de datos: un estado **y** las posiciones de los comandos que mantienen la aeronave equilibrada.

Dentro del mismo archivo del modelo de la aeronave (`jettrainer.h`) se incluyen los datos de una condición inicial que se puede obtener mediante la función `jettrainer::get_trim_condition(TrimCondition)`. La misma devuelve una estructura con los datos necesarios (`TrimPoint`) para pasarle al FDM antes de empezar a integrar. `TrimCondition` es una enumeración que permite identificar entre distintas condiciones iniciales (por ahora en el repositorio sólo se incluye una).

Tanto el estado como la posición de los comandos hay que cargarlos antes de empezar la ejecución del *loop*: el estado en el solver usando `setState()` y los comandos en `InputHandler` usando `set_controls()`.

```
#include <dlfdm/models/aircraft/jettrainer.h>
#include <dlfdm/defines.h>

...
InputHandler entradas;
...

dlfdm::AircraftParameters aermacchi_s211 = dlfdm::jettrainer::load_model();

dlfdm::FDMSolver fdm(aermacchi_s211);        // el paso de integracion se fija ACA,
                                             // y por defecto vale dt = 1/120 s

// Leer las condiciones iniciales: estado y deflexiones de comando
dlfdm::TrimPoint trim = dlfdm::jettrainer::get_trim_condition(dlfdm::jettrainer::TrimCondition::kISA5000TAS150);

fdm.setState(trim.state);                    // Cargar la condición inicial al solver
entradas.set_controls(trim.controls);        // Cargar las posiciones de comandos iniciales en InputHandler

// Dentro del loop a pasos de tiempos fijos
...
fdm.update(entradas.controls());             // El FDM se actualiza usando las posiciones
...                                          // de los comandos
```

Integración en el game loop

Finalmente, la integración de la ejecución y la lectura del estado del vuelo de la aeronave (posición y actitud) podría hacerse de la siguiente manera:

```
...
while (acc >= dt) {                     // adentro del lazo de paso fijo
    fdm.update(entradas.controls());    // avanza UN paso de dt
    acc -= dt;
}

const dlfdm::AircraftState& aircraft_state_ned = fdm.getState();   // una vez por cuadro
FlightData aircraft_data_ogl = to_world(aircraft_state_ned);       // NED -> coordenadas de la escena

escena.update(aircraft_data_ogl);
camara.update(aircraft_data_ogl.position,
              {aircraft_data_ogl.phi, aircraft_data_ogl.theta, aircraft_data_ogl.psi},
              entradas.camera_cmd());
...
```

La conversión de sistemas de referencia

El FDM utiliza el sistema de referencia NED (**N**orth **E**ast **D**own) —x al norte, y al este, z **hacia abajo**— mientras que para la escena se utiliza el sistema de OpenGL. Los dos son sistemas dextrógiros, pero con distintas convenciones para los ejes por lo que hay que hacer una transformación. Para que la terna siga siendo dextrógira hay que negar **dos** de los tres ejes, nunca uno sólo: negar uno solo produce una reflexión, con lo cual el mundo sale espejado y ninguna combinación de signos lo arregla.

```
// La transformación de sistemas de referencia debería ser única para evitar
// errores al copiar en distintos módulos
FlightData to_world(const dlfdm::AircraftState& ned)
{
    FlightData gl;
    gl.position.x =  ned.inertial_position.y;   // x_gl =  este
    gl.position.y = -ned.inertial_position.z;   // y_gl = -abajo
    gl.position.z = -ned.inertial_position.x;   // z_gl = -norte

    gl.phi   = -ned.phi;     // el menos es NOTACION, no un signo fisico:
    gl.theta =  ned.theta;   // dos de los tres ejes caen sobre semiejes
    gl.psi   = -ned.psi;     // negativos y glm::rotate se llama con ejes +

    return gl;
}
```

Cómo se sabe que está bien

- La escena se mueve a la **misma velocidad** en equipos distintos, y también al activar y desactivar la sincronización vertical.
- Sin tocar ninguna tecla, el avión **mantiene la altura y la velocidad**. Si cabecea o se va para arriba apenas arranca, falta la condición inicial completa.
- Una deflexión sostenida hace girar al avión **alrededor del eje que corresponde** y en el sentido que se seleccionó y se dejó registro, no en el contrario.
- Al **arrastrar la ventana** un par de segundos, o al volver de una pausa del depurador, la aplicación no queda congelada.
- El comando discreto cambia de estado **una vez por pulsación** y no mientras la tecla siga apretada.

Como en los prácticos anteriores, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno y hay que poder explicarlas.

La **Parte 3 no bloquea al resto del proyecto**: el terreno, el cielo y el sistema de cámaras no necesitan el modelo de vuelo andando. Lo que sí conviene tener funcionando es la **Parte 1**, porque de las nuevas funcionalidades se van a ir agregando ahí.